MSME CreditFlow AI

Architecture Document

Virtusa Jatayu Season 6 | Use Case 1: MSME Loan Underwriting Desk

Team	Tech Titans
Project type	Academic prototype / proof of concept
Technology	Python, Streamlit, SQLite, Pandas (as used in the project)
GitHub	https://github.com/vinitha-09/MSME-CreditFlow-AI
Demo	http://localhost:8501 (local computer only; not public hosting)

1. Executive Summary

MSME CreditFlow AI is a prototype for an MSME loan-underwriting desk. It demonstrates a coordinated workflow that organizes applicant information, analyses supplied transaction data, screens for suspicious indicators, estimates a transparent prototype risk result, checks illustrative policy rules, proposes indicative loan terms and generates a Credit Appraisal Memo (CAM). A central orchestrator coordinates specialist agents through a shared case record and structured messages. Calculations and rule checks are deterministic for repeatable demonstrations. The mock ledger is local and does not connect to a real bank or disburse funds.

2. Problem Statement and Objectives

MSME loan assessment can require review of bank transactions, GST/ITR information, financial statements, KYC and bureau information. Manual review may be slow and inconsistent. This prototype demonstrates a structured, evidence-oriented decision-support workflow.

Objectives:

• Consolidate case information in one place.
• Calculate basic cash-flow indicators from supplied CSV transactions.
• Highlight suspicious patterns for human investigation.
• Produce explainable prototype risk and policy results.
• Suggest indicative loan terms and generate a CAM.
• Record a proposed outcome while retaining mandatory human approval.

3. High-Level Architecture

The solution has six logical layers: (1) Presentation layer – Streamlit interface for applicant details, uploads and results; (2) Orchestration layer – coordinates the agent workflow; (3) Specialist-agent layer – document, cash-flow, fraud, credit-risk, policy and loan-structuring modules; (4) Shared case and messaging layer – carries case context and agent findings; (5) Service layer – financial calculations, CAM generation and mock-ledger operations; (6) Persistence and reporting layer – SQLite audit information, sample data and generated reports.

4. Architecture Flow

Applicant and evidence input → Streamlit UI → Orchestrator → Specialist agents → Shared case record / agent messages → CAM generation → Mock ledger proposal → Human review and approval.

The orchestrator starts the workflow and passes relevant case information to each agent. Agents return structured findings. Findings are added to the shared case record and used by later stages. Exceptions should be surfaced rather than silently ignored.

5. Specialist Agent Responsibilities

Document Intake Agent: Organizes applicant/evidence metadata and performs the document checks implemented in the prototype. Do not claim full OCR or external verification unless present in the code.
Cash-Flow Analysis Agent: Reads the supplied transaction CSV and calculates basic totals and cash-flow indicators.
Fraud Investigation Agent: Applies transparent anomaly or keyword/rule screening and returns flagged transactions for review.
Credit Risk Agent: Combines available indicators into an explainable prototype risk result; this is not a validated lending model.
Policy & Compliance Agent: Applies configured demo policy checks and identifies exceptions. Rules are illustrative, not regulatory approval.
Loan Structuring Agent: Suggests indicative loan amount/tenure/terms based on inputs and analysis; it does not approve a loan.

6. End-to-End Workflow

1. **Case intake:** User enters applicant, business and requested-loan details.
2. **Evidence upload:** User supplies transaction CSV and supporting-file references accepted by the app.
3. **Cash-flow analysis:** Transaction data is parsed and summarized using deterministic calculations.
4. **Fraud screening:** Configured rules inspect transaction descriptions/values and flag items for investigation.
5. **Credit-risk assessment:** Available metrics are converted into an explainable prototype assessment.
6. **Policy validation:** Demo policy rules identify constraints and exceptions.
7. **Loan structuring:** Indicative terms are proposed.
8. **CAM generation:** Findings and evidence are assembled into a Credit Appraisal Memo.
9. **Mock ledger:** A proposed result may be recorded in the local demo ledger.
10. **Human approval:** A reviewer must approve/reject before finalization; no autonomous disbursal occurs.

7. Data, Storage and Interfaces

Input data: Synthetic applicant information and sample bank-transaction CSV data. Supporting uploads are handled according to the current application's implemented file handling.
Storage: SQLite is used for local persistence/audit information where configured. Sample inputs are under *data/*; report-related files are under *reports/*.
Interfaces: Streamlit is the browser UI. Agent coordination and mock-ledger calls are internal application interfaces. No live bank, GST, Udyam, income-tax or credit-bureau API integration is claimed.

8. Security, Governance and Human Oversight

• Use synthetic/demo data only; do not upload real customer bank, identity or tax information to a public demo.
• Keep secrets and local environment files out of GitHub; commit only an example environment file with placeholder values.
• Validate file type, size and CSV columns before processing.
• Treat fraud flags as indicators, not proof of fraud.
• Keep policy checks explicit and reviewable.
• Require a human reviewer to approve or reject a proposed outcome.
• Never connect the mock ledger to real banking rails or initiate real disbursal.

9. Technology Stack

Python is used for application and agent logic. Streamlit provides the web interface. Pandas is used for tabular transaction processing where included. SQLite provides local persistence/audit storage. Install dependencies from *requirements.txt* and launch from the directory containing *app.py*.

10. Validation and Testing

Run the test suite from the project root with `python -m pytest -v`. Perform a manual end-to-end demo using the supplied synthetic transaction CSV: enter applicant details, upload the sample data, run analysis, inspect each agent's output, generate/download the CAM and verify the human-approval step. Record only test results that were actually observed.
Checklist:
■ App starts with `python -m streamlit run app.py` from the correct folder.
■ Sample CSV is accepted and transaction totals appear.
■ Agent outputs or clear validation errors are shown.
■ Fraud flags link to relevant evidence where available.
■ Policy exceptions are visible.
■ CAM can be generated/downloaded.
■ Mock-ledger entry is a proposal only.
■ Human approval is required.
■ No credentials, virtual environment or private customer data are committed.

11. Limitations and Future Enhancements

This is an academic proof of concept using synthetic/demo data and deterministic/rule-based analysis. It is not a production underwriting platform, validated credit-scoring model, legal/compliance engine or live banking integration. Heuristic fraud checks may produce false positives and false negatives. Supporting-file upload must not be described as OCR extraction unless verified in the source code.
Future enhancements: secure OCR with human verification; authorized GST/Udyam/bureau integrations; lender-approved policy configuration; independently validated risk models; fairness and explainability testing; role-based access; encryption; stronger audit logging and monitoring; and approved integration with bank core systems.

12. Repository and Demonstration

GitHub: https://github.com/vinitha-09/MSME-CreditFlow-AI
Local demo: http://localhost:8501. This address is accessible only on the computer running Streamlit and is not a public deployment URL. For reviewers, provide the recorded demo video or deploy through an approved hosting service.

Document note: This document is based on the project structure and information shared during development. Confirm each module description against the final source code before submission.